MAKINA COMPANY
Empresa emitente · Issuing company

Dossier Técnico da Aplicação PayInter

Pagamentos internacionais e carteira multi-moeda, com livro-razão de partidas dobradas.

Technical Dossier · Versão 1.0 · Outubro 2026

Emitido por / Issued by	**MAKINA COMPANY**
Documento / Document	**Dossier Técnico da Aplicação PayInter**
Classificação / Classification	**Confidencial — uso interno e de parceiros**
Contacto / Contact	**[EMAIL] · [TELEFONE] · [CIDADE], [PAÍS]**
Identificação fiscal / Tax ID	**[NIF]**

Índice · Contents

PARTE I — PORTUGUÊS 4

1. Sumário Executivo 5

1.1. Números do projecto 5

2. Visão Geral da Aplicação 5

2.1. Módulos funcionais 5

3. Arquitectura do Sistema 7

3.1. Organização do repositório 7

4. Modelo Financeiro — Livro-Razão de Partidas Dobradas 8

4.1. Princípios aplicados 8

4.2. Estrutura de contas 8

4.3. Garantias de implementação 9

5. Modelo de Dados 10

6. Segurança e Autenticação 11

6.1. Autenticação 11

6.2. Autorização e controlo de operações 11

7. Interface de Programação (API REST) 12

7.1. Grupos de endpoints 12

7.2. Formato de erro 12

8. Aplicação Offline-First 13

8.1. Os três mecanismos 13

8.2. O que garante a correcção do processo 13

9. Stack Tecnológico 14

10. Qualidade e Testes Automatizados 14

11. Limitações Conhecidas e Transparência Técnica 15

12. Configuração e Instalação 16

12.1. Arranque em três passos 16

12.2. Credenciais de demonstração 16

12.3. Cartões de teste (simulados) 16

12.4. Principais variáveis de configuração 16

13. Identificação da Empresa Emitente 17

PART II — ENGLISH 18

1. Executive Summary 19

2. System Architecture 19

3. Financial Model — Double-Entry Ledger 19

4. Security and Authentication 21

5. Offline-First Client 21

6. API Surface 21

7. Technology Stack 23

8. Quality Assurance 23

9. Known Limitations and Technical Transparency 23

10. Company Identification 24

PARTE I — PORTUGUÊS

Documentação técnica da aplicação PayInter, emitida pela MAKINA COMPANY como parte integrante do processo de avaliação.

1. Sumário Executivo

O **PayInter** é uma aplicação de pagamentos internacionais e carteira multi-moeda, construída como uma **prova de conceito de nível industrial**: o código não é um protótipo de ecrãs, mas um sistema com livro-razão financeiro de partidas dobradas, autenticação por palavra-passe e PIN, controlo de acessos por função (RBAC), trilha de auditoria e uma aplicação cliente que continua a funcionar sem ligação à Internet.

A aplicação está dividida em três componentes independentes: uma **API REST** em Node.js e TypeScript, uma **base de dados relacional** PostgreSQL com migrações versionadas, e uma **aplicação cliente** do tipo PWA (*Progressive Web App*) instalável, que pode ser empacotada como aplicação Android (APK) através do Capacitor.

1.1. Números do projecto

Métrica	Valor	Nota
Endpoints REST implementados	95	Distribuídos por 18 grupos de rotas
Tabelas na base de dados	25	Migrações SQL versionadas
Linhas de código — servidor	≈ 6 100	Rotas, serviços, bibliotecas, middleware
Linhas de código — cliente	≈ 2 830	17 ecrãs, sincronização offline
Testes automatizados	34	5 ficheiros — 100% a passar
Dependências de execução	8	Servidor: Express, zod, bcrypt, JWT, PGlite/pg
Instalação do cliente	PWA + APK	Empacotamento Android via Capacitor

> **Aviso essencial — ambiente de demonstração**
> O PayInter está configurado, por omissão, como **sandbox** (ambiente de teste controlado). Não movimenta dinheiro real, não cobra cartões reais e **não possui licença de instituição financeira**. Todos os fornecedores — processamento de cartões, câmbios, verificação de identidade (KYC) e transferências internacionais — são **simulados**. Qualquer utilização com fundos reais exige licenciamento, contratos com fornecedores e parecer jurídico prévio. Ver capítulo 11.

2. Visão Geral da Aplicação

O PayInter foi desenhado para o problema das transferências internacionais e pagamentos digitais em mercados onde a cobertura bancária é desigual e a ligação à Internet é intermitente — um contexto que descreve com precisão Angola e a sua diáspora. A aplicação responde a esse contexto com duas decisões estruturais:

- **Funciona offline primeiro.** O cliente guarda uma cópia local dos dados do utilizador e enfileira as operações feitas sem rede, reenviando-as automaticamente quando a ligação regressa — sem duplicar pagamentos.
- **O saldo não é um número guardado: é um resultado calculado.** Toda a movimentação de valor passa por um livro-razão de partidas dobradas, o que permite detectar e corrigir divergências em vez de as esconder.

2.1. Módulos funcionais

Módulo	O que faz	Estado
Carteira multi-moeda	Saldos por divisa (USD, EUR, GBP, CAD, BRL, AOA, ZAR), extracto e detalhe de movimentos	Implementado
Transferências P2P	Envio entre utilizadores por identificador PayTag, com taxas e verificação de saldo	Implementado
Pagamentos QR	Geração de código assinado (HMAC) e pagamento por leitura	Implementado

Módulo	O que faz	Estado
Pagamentos a terceiros	Carregamento de saldo, transferência bancária e carteira móvel	Implementado
Cartões	Emissão de cartão virtual, congelamento, descongelamento e cartão predefinido	Implementado
Transferências internacionais	Cotação de câmbio, destinatários e envio internacional	Implementado
Verificação de identidade (KYC)	Submissão de documentos e revisão por operador	Implementado
Notificações	Lista, marcação de leitura e preferências	Implementado
Apoio ao cliente	Pedidos de assistência, mensagens e perguntas frequentes	Implementado
Segurança da conta	PIN, código de uso único (OTP), sessões activas e histórico de eventos	Implementado
Consola de operações	Painel administrativo com função e permissões	API pronta; interface web ainda não incluída
Privacidade	Exportação de dados pessoais e extractos de cartão	Implementado

3. Arquitectura do Sistema

A arquitectura segue uma separação estrita em camadas. O cliente nunca escreve directamente na base de dados: toda a mutação passa por um endpoint validado, que por sua vez passa por um serviço de domínio, e só depois pelo livro-razão.

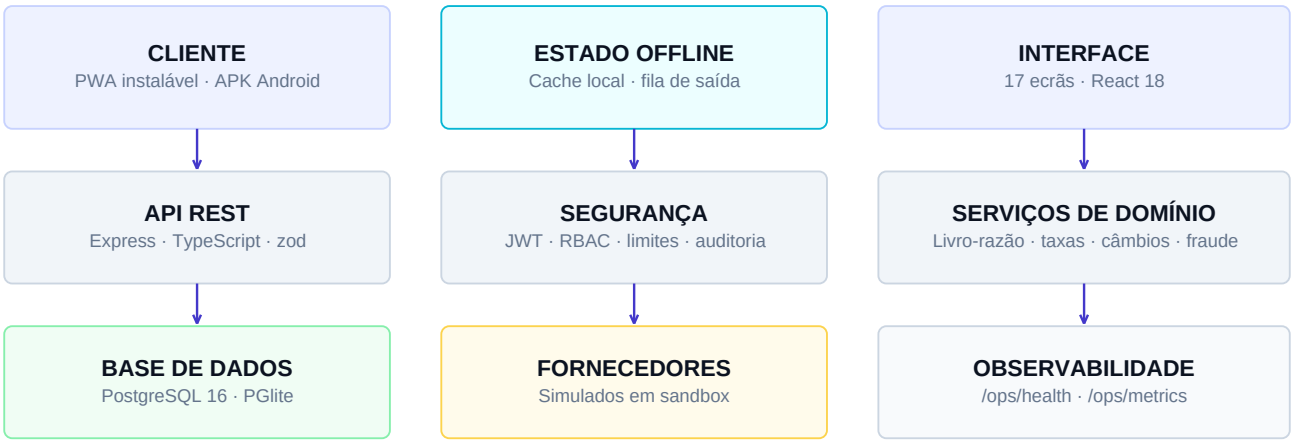


Figura 1 — Camadas do sistema. As setas indicam a direcção das chamadas: o cliente comunica exclusivamente com a API; a API é a única porta de entrada para a base de dados.

3.1. Organização do repositório

Pasta	Conteúdo
database/migrations/	Esquema SQL versionado (0001 a 0003). É a fonte de verdade da estrutura de dados.
backend/src/routes/	18 grupos de rotas: autenticação, carteira, transferências, pagamentos, cartões, câmbios, QR, KYC, apoio, privacidade, administração.
backend/src/services/	Lógica de domínio: livro-razão, carteira, taxas, câmbios, fraude, auditoria, notificações.
backend/src/middleware/	Autenticação, autorização por função, limites de pedidos, idempotência, tratamento de erros.
backend/tests/	34 testes automatizados executados contra uma base de dados real em memória.
mobile/	Aplicação cliente: 17 ecrãs, gestão de sessão, cache local, fila de operações offline e *service worker*.

4. Modelo Financeiro — Livro-Razão de Partidas Dobradas

Esta é a decisão técnica mais importante do projecto e a que mais distingue o PayInter de uma simples maqueta de aplicação de pagamentos.

Num sistema ingénuo, o saldo de um utilizador é um número guardado numa tabela e actualizado com `UPDATE saldo = saldo - 100`. Esse desenho falha silenciosamente: um erro, uma repetição de pedido ou uma corrida entre dois processos deixa o dinheiro a existir ou a faltar sem que ninguém saiba. No PayInter, **o saldo nunca é alterado directamente**.

4.1. Princípios aplicados

- **Toda a movimentação de valor é uma transacção contabilística.** Cada operação gera um registo em `ledger_transactions` e um conjunto de movimentos em `ledger_entries`, cada um com débito ou crédito.
- **O total dos débitos iguala o total dos créditos.** Esta igualdade é a invariante do sistema: se se quebrar, há um erro detectável.
- **Os saldos visíveis são uma cache derivada.** A tabela de saldos existe por desempenho, mas pode ser reconstruída a partir do livro-razão a qualquer momento.
- **Existe verificação automática de consistência.** O serviço de reconciliação compara a cache com a soma das entradas e sinaliza divergências.
- **Cada movimento guarda o saldo resultante.** O campo `balance_after_minor` permite auditar o histórico sem recalcular nada.

4.2. Estrutura de contas

As contas seguem uma convenção de nomenclatura que identifica o proprietário e a divisa, e que separa as contas de utilizador das contas do sistema:

Padrão de conta	Tipo	Significado
WALLET:{userId}:{DIVISA}	Utilizador	Conta individual de cada utilizador por divisa (ex.: WALLET:u_8f3…:USD)
SYS:clearing:{DIVISA}	Sistema	Conta de compensação — detém o valor durante o processamento
SYS:settlement:{DIVISA}	Sistema	Liquidação com o exterior
SYS:revenue:{DIVISA}	Sistema	Receita de taxas cobradas pela plataforma
SYS:suspense:{DIVISA}	Sistema	Valor retido para revisão manual (disputas, suspeitas de fraude)

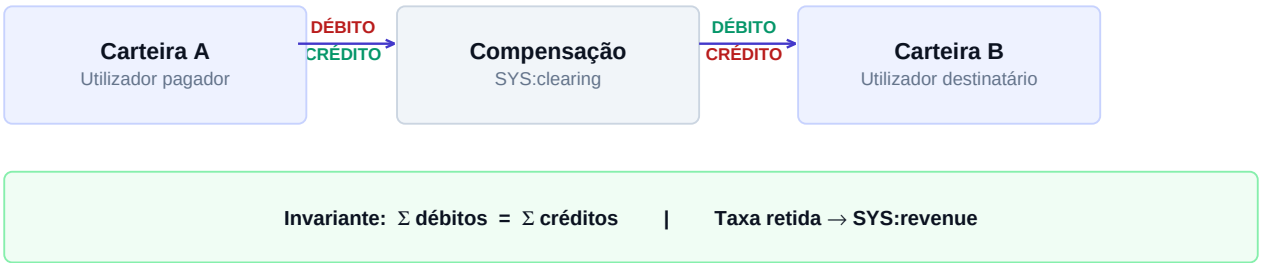


Figura 2 — Percurso do valor numa transferência entre utilizadores. Cada movimento tem uma contrapartida; a soma dos débitos iguala sempre a soma dos créditos.

4.3. Garantias de implementação

- **Operações atómicas.** A transacção contabilística e todos os seus movimentos são escritos dentro de uma única transacção de base de dados: ou ficam todos, ou não fica nenhum.
- **Pedidos repetidos não duplicam dinheiro.** Cada pedido de movimentação carrega uma chave de idempotência (`Idempotency-Key`); a repetição devolve o resultado original em vez de cobrar duas vezes.
- **Reconciliação sob pedido.** O serviço compara cada saldo em cache com o somatório do livro-razão, detecta transacções cujos totais não batem certo e devolve um relatório com os problemas encontrados.
- **Reconstrução possível.** Como o livro-razão é a fonte de verdade, é sempre possível reconstruir os saldos e verificar toda a história financeira.

5. Modelo de Dados

A base de dados tem 25 tabelas, organizadas por domínio funcional. O esquema é versionado em três migrações SQL, aplicadas automaticamente na primeira execução, pelo que não existe passos manuais de instalação.

Domínio	Tabelas	Finalidade
Identidade e acesso	users · user_sessions · otp_codes · admin_users · security_events	Contas, sessões activas, códigos de uso único, operadores e registo de eventos de segurança
Livro-razão	ledger_accounts · ledger_transactions · ledger_entries	O núcleo financeiro: contas, transacções contabilísticas e movimentos de débito/crédito
Saldos e carteiras	wallets · wallet_balances	Carteira por utilizador e cache de saldos (derivada do livro-razão)
Movimentos	transactions · idempotency_keys	Registo funcional dos movimentos com estado, e chaves que impedem duplicação
Pagamentos e cartões	payment_cards · recipients	Cartões virtuais, destinatários frequentes e dados de envio
Câmbios e taxas	currencies · fx_rates · fee_rules · provider_configs	Divisas suportadas, taxas de câmbio, regras de comissão e configuração de fornecedores
Cumprimento e risco	kyc_submissions · fraud_alerts	Processos de verificação de identidade e alertas automáticos de fraude
Comunicação e apoio	notifications · notification_prefs · support_tickets · support_messages	Notificações, preferências do utilizador e sistema de apoio ao cliente
Auditoria	audit_logs	Registo imutável de acções sensíveis: administração e operações com dinheiro

Integridade referencial e limites
As tabelas usam chaves estrangeiras, restrições de verificação e índices parciais (por exemplo, sobre movimentos em estado pendente). Os valores monetários são guardados como **inteiros na unidade mínima** da divisa (cêntimos), nunca como números decimais — o que elimina os erros de arredondamento típicos de cálculos com vírgula flutuante.

6. Segurança e Autenticação

A segurança é tratada em profundidade, em várias camadas independentes. Nenhuma delas é suficiente por si só; em conjunto, tornam o acesso indevido e a duplicação de operações substancialmente mais difíceis.

6.1. Autenticação

Mecanismo	Implementação
Palavra-passe	Guardada com bcrypt (algoritmo de derivação lento e com sal). A palavra-passe original nunca é armazenada nem registada.
PIN da aplicação	Derivado com bcrypt e reforçado com um "pepper" secreto guardado fora da base de dados, de modo a que a fuga da base de dados, por si só, não exponha os PINs.
Sessão de acesso	Token JWT assinado, válido por 15 minutos.
Sessão de renovação	Token aleatório de 48 bytes, válido por 30 dias, guardado apenas como resumo criptográfico (SHA-256). É **rodado a cada renovação**: um token usado nunca volta a ser aceite.
Revogação	Encerramento de sessão e listagem de dispositivos activos com possibilidade de revogação individual.
Código de uso único (OTP)	6 dígitos, associados ao utilizador, válidos por 10 minutos, com máximo de 5 tentativas e guardados apenas como resumo.
Limites de pedidos	Autenticação: 20 pedidos por minuto por origem. OTP: 5 pedidos por minuto por origem e conta.

6.2. Autorização e controlo de operações

- **Perfis de acesso aplicados no servidor.** As permissões dos operadores são verificadas em cada pedido, no servidor. Ocultar botões no cliente não é tratado como medida de segurança.
- **Registo de auditoria.** As acções sensíveis — administração, ajustes, reembolsos, alterações de estado — ficam registadas com autor, data e detalhe.
- **Validação de todas as entradas.** Os pedidos são validados contra esquemas formais (zod) antes de chegarem à lógica de negócio.
- **Cabeçalhos de segurança.** Aplicados por omissão através do Helmet, com lista de origens permitidas para pedidos entre domínios.
- **Segredos fora do código.** Chaves, senhas e credenciais são lidas apenas de variáveis de ambiente. O repositório contém ficheiros de exemplo sem valores reais.
- **Detecção de fraude.** Regras automáticas sinalizam montantes elevados e frequência anómala de operações, colocando-as em fila de revisão manual em vez de as executar cegamente.

> **Antes de qualquer utilização com fundos reais**
> A configuração de produção exige substituir todas as chaves de exemplo, activar HTTPS obrigatório, ligar um serviço real de gestão de segredos, definir retenção e cópia de segurança da base de dados, e obter parecer jurídico sobre o enquadramento aplicável. O estado actual é adequado a demonstração e avaliação técnica, não a operação com dinheiro real.

7. Interface de Programação (API REST)

A API expõe 95 endpoints organizados por domínio, todos sob o prefixo `/v1`. A autenticação é feita por token Bearer no cabeçalho do pedido. As respostas de erro seguem um formato único, o que simplifica o tratamento no cliente.

7.1. Grupos de endpoints

Grupo	Prefixo	Endpoints	Destaques
Autenticação	`/v1/auth`	12	registo, sessão, renovação, PIN, OTP, sessões activas
Carteira	`/v1/wallet`	1	saldos consolidados por divisa
Utilizadores	`/v1/users`	1	pesquisa por PayTag para envio entre utilizadores
Transferências	`/v1/transfers`	1	envio entre utilizadores com taxa
Movimentos	`/v1/transactions`	3	extracto, detalhe e abertura de disputa
Pagamentos	`/v1/payments`	4	carregamento, banco e carteira móvel
Carregamentos	`/v1/topups`	2	cotação e execução
Cartões	`/v1/cards`	5	emissão, congelar, descongelar, predefinido
Internacional	`/v1/payments/international`	1	envio com câmbio
Destinatários	`/v1/recipients`	4	gestão de contactos de envio
Códigos QR	`/v1/qr`	3	gerar, resolver e pagar
Verificação (KYC)	`/v1/kyc`	2	submissão e estado
Câmbios	`/v1/fx`	2	taxas e cotação
Notificações	`/v1/notifications`	5	lista, leitura e preferências
Apoio	`/v1/support`	5	pedidos, mensagens e FAQ
Privacidade	`/v1/privacy`	2	exportação de dados e extractos
Jurídico	`/v1/legal`	2	termos, privacidade e avisos legais
Administração	`/v1/admin`	38	painel de operações, revisões, risco, taxas e auditoria
Operação técnica	`/ops`	3	estado, prontidão e métricas

7.2. Formato de erro

Todos os erros devolvem a mesma estrutura, com um código estável que o cliente pode interpretar sem depender do texto da mensagem:

```
{
  "error": {
    "code": "INSUFFICIENT_FUNDS",
    "message": "Saldo insuficiente para esta operação",
    "details": { "availableMinor": 1250, "requiredMinor": 5000 }
  }
}
```

8. Aplicação Offline-First

O cliente foi construído para o cenário real de rede intermitente: em vez de mostrar um ecrã de erro quando não há ligação, continua a funcionar e sincroniza depois. Há três mecanismos distintos a cooperar.

8.1. Os três mecanismos

Mecanismo	Como funciona
Instalação e arranque	Um *service worker* guarda a estrutura da aplicação (ecrãs, estilos, código) no dispositivo. A aplicação abre instantaneamente e funciona sem rede.
Leitura de dados	As leituras tentam primeiro a rede. Se falharem, é devolvida a última cópia guardada localmente, por utilizador, no IndexedDB do navegador. O utilizador vê os seus dados com um aviso claro de que podem estar desactualizados.
Escrita de dados	Uma operação feita sem rede é guardada numa **fila de saída local**, com uma chave de idempotência própria. O interface mostra-a imediatamente como pendente. Quando a ligação regressa, a fila é enviada pela ordem original.



Figura 3 — Os dois caminhos do cliente offline. A chave de idempotência é o elemento que garante que a sincronização não duplica pagamentos.

8.2. O que garante a correcção do processo

- **Sem duplicação.** Mesmo que a aplicação seja encerrada a meio do reenvio, ou que o mesmo pedido chegue duas vezes ao servidor, a chave de idempotência garante uma única execução. Este é o ponto onde aplicações offline costumam falhar, e é tratado explicitamente.
- **Ordem preservada.** As operações são reenviadas pela ordem em que foram criadas, evitando que um pagamento posterior seja aplicado antes do anterior.
- **Estado visível.** O utilizador vê sempre o que está pendente, o que falhou e o motivo da rejeição.
- **Fila limitada.** As operações com erro definitivo (por exemplo, saldo insuficiente) são retiradas da fila com explicação, em vez de a bloquearem.

9. Stack Tecnológico

As tecnologias foram escolhidas por maturidade e por facilidade de contratação em contexto angolano, evitando dependências exóticas de difícil manutenção local.

Componente	Tecnologia	Versão	Porquê
Servidor	Node.js + TypeScript	≥ 20	Uma só linguagem no servidor e no cliente; tipos verificados em compilação
Framework HTTP	Express	4.21	Maduro, documentado, fácil de manter
Validação	zod	3.23	Validação formal das entradas dos pedidos
Base de dados	PostgreSQL	16	Integridade transaccional e consultas fiáveis
Base de dados (desenvolvimento)	PGlite	0.2	PostgreSQL embedding em memória — permite arrancar e testar sem instalar nada
Palavras-passe e PINs	bcryptjs	2.4	Derivação lenta resistente a ataques por força bruta
Sessões	jsonwebtoken	9.0	Tokens de acesso assinados
Interface	React	18.3	Ecossistema estável e ampla base de programadores
Construção	Vite	5.4	Compilação rápida e empacotamento em PWA
Aplicação instalável	vite-plugin-pwa + Workbox	0.21 / 7.3	Instalação e funcionamento offline
Empacotamento Android	Capacitor	6.2	Gera o APK a partir do mesmo código do cliente
Testes	Vitest	2.1	Testes unitários e de integração sobre a API real

10. Qualidade e Testes Automatizados

Existem 34 testes automatizados, distribuídos por cinco áreas, e todos passam no estado actual do código. Os testes correm contra uma base de dados PostgreSQL real criada em memória, e não contra simulações, o que aumenta significativamente a sua capacidade de detectar problemas reais.

Ficheiro de teste	Testes	Cobertura
tests/ledger.test.ts	5	Livro-razão: equilíbrio de débitos e créditos, saldos, reconciliação, taxas
tests/payments.test.ts	9	Pagamentos, carregamentos, transferências e tratamento de saldo insuficiente
tests/auth.test.ts	6	Registo, sessão, renovação de token, PIN e códigos de uso único
tests/admin.test.ts	8	Permissões de operador, revisões de KYC, reembolsos e auditoria
tests/qr_intl.test.ts	6	Códigos QR assinados e transferências internacionais
Total	34	100% a passar

11. Limitações Conhecidas e Transparência Técnica

Um documento técnico honesto deve declarar o que **não** está feito. Os pontos seguintes são limitações reais do estado actual do projecto e devem ser considerados em qualquer decisão:

Ponto	Situação real	Impacto
Ambiente de demonstração	Por omissão, todos os fornecedores são simulados: cartões, câmbios, KYC e transferências internacionais.	Nenhum valor real é movimentado. É adequado para demonstração e avaliação, não para operação real.
Licenciamento	O projecto não possui licença de instituição financeira ou de prestador de serviços de pagamento.	Bloqueia por completo qualquer utilização com fundos reais. Requer enquadramento junto do regulador competente.
Consola de operações	A API administrativa está implementada (38 endpoints), mas a interface web para operadores não está incluída no repositório.	A gestão de utilizadores, revisões e risco é feita por chamada directa à API.
Documentação de referência	O ficheiro de apresentação do repositório menciona uma pasta de consola de administração, documentação de arquitectura e ficheiros de orquestração de contentores que ainda não existem no código.	Divergência entre a descrição e o conteúdo real. Este dossier reflecte o estado efectivo do código.
Número de testes	A apresentação do repositório refere 52 testes; o código contém 34.	Discrepância documental, sem impacto funcional, mas relevante para avaliação técnica.
Distribuição Android	O processo automático de geração do APK está preparado, mas ainda não foi executado com sucesso nenhuma vez.	O APK ainda não existe como artefacto verificável.
Segurança em produção	Chaves de exemplo, ausência de gestor de segredos, cópias de segurança e monitorização não configuradas.	Requer trabalho antes de qualquer exposição a tráfego real.

Recomendação da Makina Company

Recomendamos apresentar o PayInter como **prova de conceito técnica avançada** e não como produto em operação. É um activo real e demonstrável: o livro-razão, o modelo offline e a API são trabalho efectivo e verificável. A linguagem de comunicação deve acompanhar essa realidade — a credibilidade técnica é precisamente o argumento mais forte deste projecto.

12. Configuração e Instalação

O sistema arranca sem infraestrutura externa: a base de dados é criada em memória, as migrações e os dados de demonstração são aplicados automaticamente. Isto permite demonstrar o sistema completo a partir de um único comando.

12.1. Arranque em três passos

```
# 1. Servidor de aplicação (cria base de dados, migrações e dados de demonstração)
cd backend  &&  npm install  &&  npm run dev
   → http://localhost:4000/ops/health

# 2. Aplicação cliente (PWA instalável)
cd mobile   &&  npm install  &&  npm run dev
   → http://localhost:5173

# 3. Verificação automática
cd backend  &&  npm test              # 34 testes
```

12.2. Credenciais de demonstração

Contexto	Utilizador	Palavra-passe	Extras
Aplicação cliente	demo@payinter.app	Demo1234!	PIN 2468 · código OTP 123456
Administração	admin@payinter.app	Admin1234!	Perfil de superadministrador

12.3. Cartões de teste (simulados)

Número do cartão	Comportamento esperado
4242 4242 4242 4242	Pagamento aceite
4000 0000 0000 9995	Pagamento recusado
4000 0025 0000 3155	Requer confirmação adicional (3-D Secure)

12.4. Principais variáveis de configuração

Todas as credenciais são lidas de variáveis de ambiente; o repositório contém apenas ficheiros de exemplo sem valores reais.

Variável	Função
NODE_ENV / PORT	Ambiente de execução e porta do servidor
DEMO_MODE / SANDBOX_PROVIDERS	Activa o modo demonstração e os fornecedores simulados
DATABASE_URL / DB_EMBEDDED	Ligação a PostgreSQL ou uso da base de dados em memória
JWT_SECRET / JWT_REFRESH_SECRET	Chaves de assinatura das sessões
PIN_PEPPER / QR_SIGNING_SECRET	Reforço do PIN e assinatura dos códigos QR
ACCESS_TOKEN_TTL_SECONDS	Validade do token de acesso (por omissão, 900 s)
OTP_TTL_SECONDS / OTP_MAX_ATTEMPTS	Validade e tentativas dos códigos de uso único
FRAUD_LARGE_AMOUNT_MINOR	Limite a partir do qual um movimento é sinalizado
CORS_ORIGINS	Origens autorizadas a comunicar com a API

13. Identificação da Empresa Emitente

Este documento foi preparado e emitido pela MAKINA COMPANY, como parte da avaliação técnica e da preparação de apresentação da aplicação PayInter.

Campo	Valor
Empresa	MAKINA COMPANY
Documento	Dossier Técnico da Aplicação PayInter
Versão do documento	1.0
Data de emissão	Outubro de 2026
Estado do software	Prova de conceito — ambiente de demonstração (sandbox)
Responsável técnico	[NOME DO RESPONSÁVEL]
Endereço	[ENDEREÇO], [CIDADE], [PAÍS]
Contacto telefónico	[TELEFONE]
Correio electrónico	[EMAIL]
Sítio na Internet	[SITE]
Identificação fiscal	[NIF]
Registo comercial	[REGISTO COMERCIAL]

Nota sobre os dados em falta
Os campos indicados entre parênteses rectos estão deliberadamente por preencher. Devem ser substituídos pelos dados oficiais da empresa antes de qualquer divulgação externa, impressão ou envio a terceiros.

Documento confidencial. A informação técnica aqui descrita foi extraída directamente do código-fonte do projecto PayInter no estado em que se encontrava à data de emissão. O sistema funciona, por omissão, em ambiente de demonstração e não movimenta fundos reais nem se apresenta como entidade licenciada.

PART II — ENGLISH

Technical documentation of the PayInter application, issued by MAKINA COMPANY as part of the evaluation process.

1. Executive Summary

PayInter is an international payments and multi-currency wallet application built as an **industrial-grade proof of concept**: the codebase is not a screen prototype but a system with a double-entry financial ledger, password and PIN authentication, role-based access control, an audit trail, and a client application that keeps working without an Internet connection.

The application is split into three independent components: a **REST API** in Node.js and TypeScript, a **relational database** (PostgreSQL) with versioned migrations, and an installable **client application** (PWA) that can be packaged as an Android application (APK) through Capacitor.

Metric	Value	Note
REST endpoints implemented	95	Across 18 route groups
Database tables	25	Versioned SQL migrations
Backend lines of code	≈ 6,100	Routes, services, libraries, middleware
Client lines of code	≈ 2,830	17 screens, offline synchronisation
Automated tests	34	5 test files — all passing
Client distribution	PWA + APK	Android packaging via Capacitor

> **Essential notice — demonstration environment**
> PayInter is configured by default as a **sandbox**. It does not move real money, does not charge real cards, and **holds no financial institution licence**. All providers — card processing, foreign exchange, identity verification (KYC) and international transfers — are **simulated**. Any use with real funds requires licensing, provider contracts and prior legal advice. See chapter 11.

2. System Architecture

The architecture follows a strict layer separation. The client never writes to the database directly: every mutation goes through a validated endpoint, then through a domain service, and only then through the ledger.

Layer	Responsibility
Client (PWA / APK)	17 React screens, session handling, offline cache and local outbox
API (Express + TypeScript)	Request validation, authentication, authorisation, rate limiting, idempotency
Domain services	Ledger postings, fees, FX quotes, fraud rules, audit logging
Data layer	PostgreSQL 16 (production) or PGlite in-memory (development and tests)
Providers	Simulated in sandbox: card processor, FX feed, KYC and payouts
Observability	`/ops/health, /ops/ready and /ops/metrics endpoints`

3. Financial Model — Double-Entry Ledger

This is the most important technical decision in the project and the clearest distinction between PayInter and a simple payments mock-up.

In a naive system, a user balance is a stored number updated with `UPDATE balance = balance - 100`. That design fails silently: an error, a retried request or a race between two processes leaves money appearing or vanishing with nobody aware. In PayInter, **a balance is never mutated directly**.

- **Every value movement is an accounting transaction.** Each operation creates a record in `ledger_transactions` plus a set of legs in `ledger_entries`, each a debit or a credit.
- **Total debits always equal total credits.** This equality is the system invariant: if it breaks, an error is detectable.
- **Visible balances are a derived cache.** The balance table exists for performance but can be rebuilt from the ledger at any time.
- **Consistency is checked automatically.** A reconciliation service compares the cache against the sum of ledger entries and reports any divergence.
- **Every leg stores the resulting balance.** The `balance_after_minor` field allows historical auditing without recomputation.

Account pattern	Type	Meaning
WALLET:{userId}:{CCY}	User	Per-user account for each currency
SYS:clearing:{CCY}	System	Clearing account holding value while processing
SYS:settlement:{CCY}	System	Settlement with the outside world
SYS:revenue:{CCY}	System	Platform fee revenue
SYS:suspense:{CCY}	System	Value held for manual review (disputes, fraud suspicion)

Correctness guarantees in the implementation

Atomicity: a ledger transaction and all its legs are written inside a single database transaction — all or nothing. **No double spending on retries:** every mutation carries an Idempotency-Key; a repeat returns the original result instead of charging twice. **Reconstructible:** since the ledger is the source of truth, balances and full financial history can always be rebuilt and audited.

4. Security and Authentication

Mechanism	Implementation
Password	Stored with bcrypt (salted, deliberately slow). The original password is never stored or logged.
Application PIN	bcrypt-derived and strengthened with a secret "pepper" held outside the database, so a database leak alone does not expose PINs.
Access session	Signed JWT, valid for 15 minutes.
Refresh session	Random 48-byte token, valid for 30 days, stored only as a SHA-256 digest. **Rotated on every renewal**: a used token is never accepted again.
One-time codes (OTP)	6 digits, user-bound, 10-minute expiry, maximum 5 attempts, stored only as a digest.
Rate limiting	Authentication: 20 requests/minute per origin. OTP: 5 requests/minute per origin and account.
Authorisation	Role permissions enforced server-side on every request; hiding client buttons is not treated as a security control.
Audit trail	Sensitive actions — administration, adjustments, refunds, status changes — are logged with actor, timestamp and detail.
Secrets	Read exclusively from environment variables; the repository holds example files with no real values.
Fraud detection	Automatic rules flag large amounts and anomalous transaction frequency, routing them to manual review instead of executing blindly.

5. Offline-First Client

The client is built for the real scenario of intermittent connectivity: instead of showing an error screen when the network is unavailable, it keeps working and synchronises afterwards. Three mechanisms cooperate:

Mechanism	How it works
Installation and start-up	A service worker caches the application shell (screens, styles, code). The app opens instantly and works offline.
Reading data	Reads try the network first and fall back to the last locally cached copy, per user, in IndexedDB — with a clear staleness warning.
Writing data	An operation made offline is stored in a local outbox with its own idempotency key and shown immediately as pending. When connectivity returns, the queue is replayed in original order.

The idempotency key is the element that guarantees synchronisation never duplicates a payment — even if the app is closed mid-replay or the same request reaches the server twice. This is where offline-first applications most commonly fail, and it is handled explicitly here.

6. API Surface

95 endpoints are exposed under the `/v1` prefix, authenticated with a Bearer token. Errors share a single stable shape:

```
{
  "error": {
    "code": "INSUFFICIENT_FUNDS",
    "message": "Insufficient balance for this operation",
    "details": { "availableMinor": 1250, "requiredMinor": 5000 }
  }
}
```

Group	Prefix	Endpoints
Authentication	`/v1/auth`	12
Wallet and users	`/v1/wallet · /v1/users`	2
Transfers and transactions	`/v1/transfers · /v1/transactions`	4
Payments and top-ups	`/v1/payments · /v1/topups`	6
Cards	`/v1/cards`	5
International and recipients	`/v1/payments/international · /v1/recipients`	5
QR codes	`/v1/qr`	3
KYC, FX and notifications	`/v1/kyc · /v1/fx · /v1/notifications`	9
Support, privacy and legal	`/v1/support · /v1/privacy · /v1/legal`	9
Administration	`/v1/admin`	38
Operations	`/ops`	3

7. Technology Stack

Component	Technology	Version
Runtime	Node.js + TypeScript	≥ 20
HTTP framework	Express	4.21
Validation	zod	3.23
Database	PostgreSQL	16
Embedded database (dev/tests)	PGlite	0.2
Passwords and PINs	bcryptjs	2.4
Sessions	jsonwebtoken	9.0
UI	React	18.3
Build tool	Vite	5.4
Installable app / offline	vite-plugin-pwa + Workbox	0.21 / 7.3
Android packaging	Capacitor	6.2
Testing	Vitest	2.1

8. Quality Assurance

34 automated tests run against a real in-memory PostgreSQL instance rather than mocks, which substantially increases their ability to catch real defects. All pass on the current codebase.

Test file	Tests	Coverage
tests/ledger.test.ts	5	Ledger balance invariants, balances, reconciliation, fees
tests/payments.test.ts	9	Payments, top-ups, transfers, insufficient-funds handling
tests/auth.test.ts	6	Registration, sessions, token renewal, PIN, OTP
tests/admin.test.ts	8	Operator permissions, KYC review, refunds, audit logging
tests/qr_intl.test.ts	6	Signed QR codes and international transfers
Total	34	All passing

9. Known Limitations and Technical Transparency

An honest technical document must state what is **not** finished. The following are real limitations of the current state of the project:

Item	Actual situation	Impact
Demonstration environment	All providers are simulated by default: cards, FX, KYC and international transfers.	No real value moves. Suitable for demonstration and technical evaluation, not for live operation.
Licensing	The project holds no financial institution or payment service provider licence.	Completely blocks any use with real funds; requires engagement with the competent regulator.
Operations console	The administrative API is implemented (38 endpoints) but the operator web interface is not included in the repository.	User management, reviews and risk are handled through direct API calls.

Item	Actual situation	Impact
Reference documentation	The repository presentation mentions an administration console folder, architecture documentation and container orchestration files that do not yet exist in the code.	Divergence between the description and the actual content. This dossier reflects the effective state of the code.
Test count	The repository presentation states 52 tests; the code contains 34.	Documentation discrepancy with no functional impact, but relevant to technical review.
Android distribution	The automated APK build pipeline is prepared but has never completed successfully.	No APK exists yet as a verifiable artefact.
Production security	Example keys, no secrets manager, no backups and no monitoring.	Work required before any exposure to real traffic.

Recommendation
We recommend presenting PayInter as an **advanced technical proof of concept** rather than a live product. It is a real, demonstrable asset: the ledger, the offline model and the API are genuine, verifiable engineering. Communication should match that reality — technical credibility is precisely this project's strongest argument.

10. Company Identification

This document was prepared and issued by MAKINA COMPANY as part of the technical evaluation and presentation preparation for the PayInter application.

Field	Value
Company	MAKINA COMPANY
Document	PayInter Application — Technical Dossier
Document version	1.0
Issue date	October 2026
Software status	Proof of concept — demonstration (sandbox) environment
Technical lead	[NAME]
Address	[ADDRESS], [CITY], [COUNTRY]
Phone	[PHONE]
Email	[EMAIL]
Website	[WEBSITE]
Tax identification	[TAX ID]
Commercial registration	[COMMERCIAL REGISTRATION]

Confidential document. All technical information described here was extracted directly from the PayInter source code as it stood on the issue date. The system operates by default in a demonstration environment; it does not move real funds and does not present itself as a licensed entity.